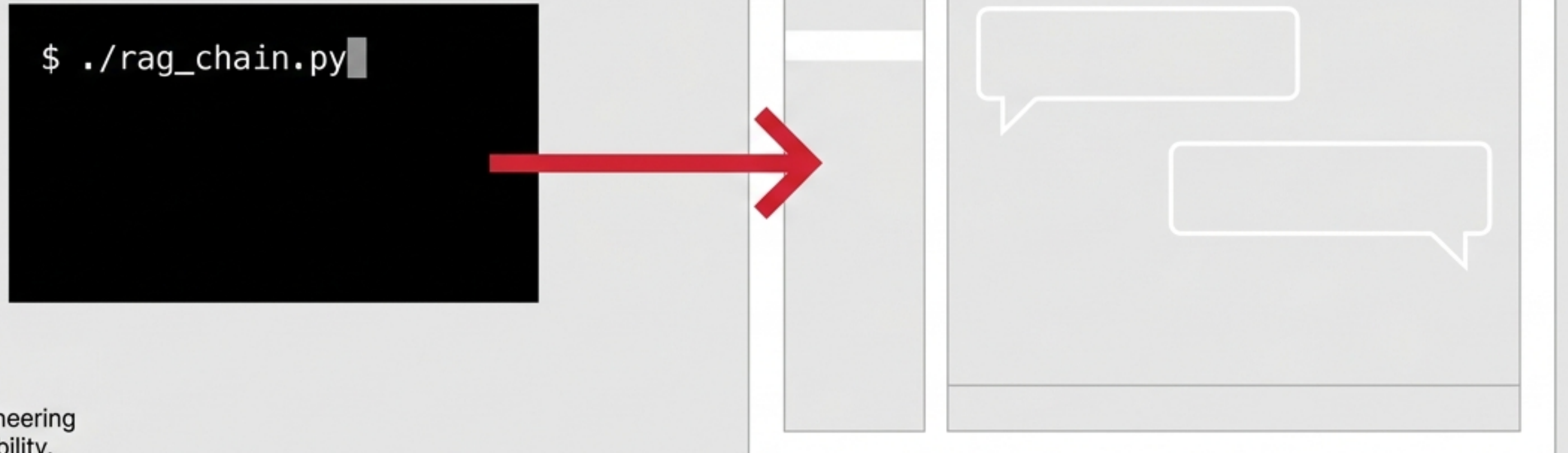


Phase 3: Building the Web Interface

From Terminal Script to
Browser Application



Bridging the gap between engineering
infrastructure and user accessibility.

Taking the System Public

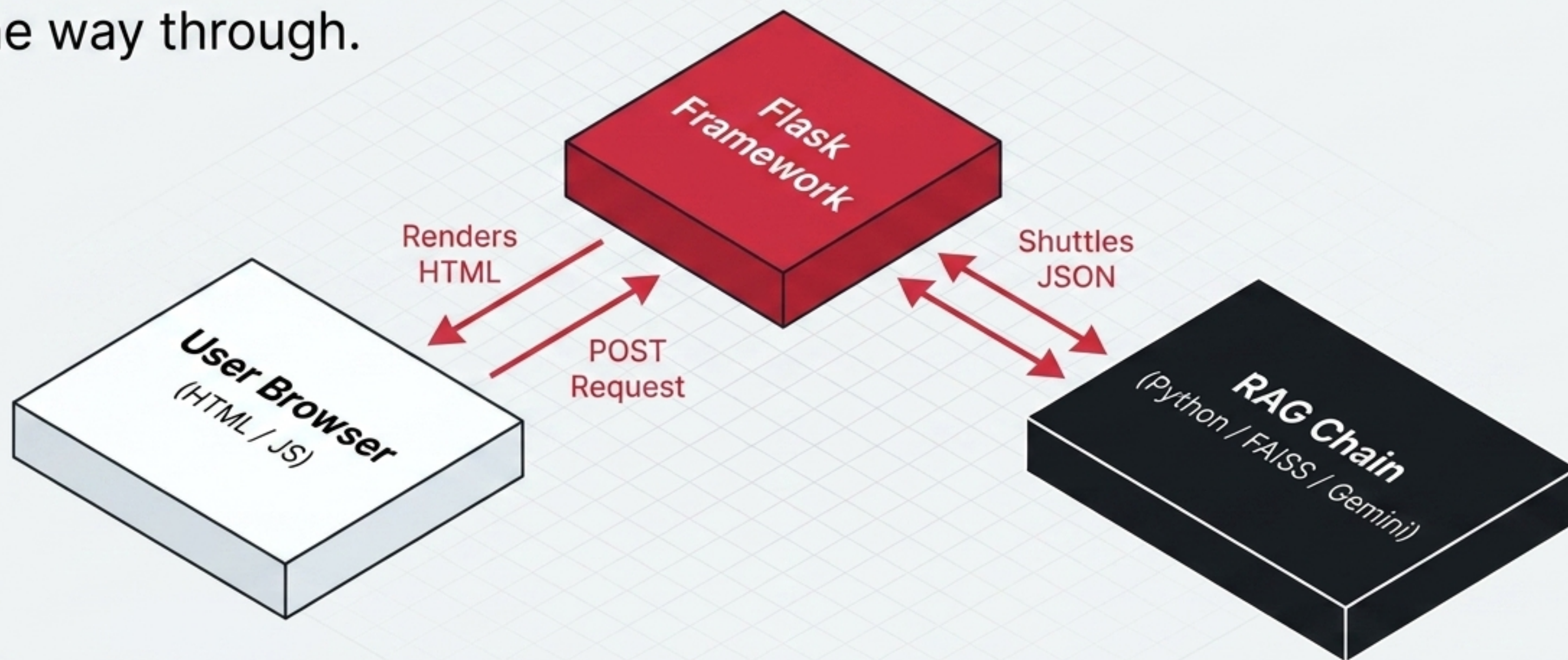
A working script becomes a widely accessible tool.

Phases 1 & 2	Phase 3
	
Developers Only	Accessible to Anyone
Code Execution Required	Plain English Input
Local Terminal Interface	Shared URL Access

No technical knowledge, terminal access, or code required by the end-user.

Flask as the Bridge

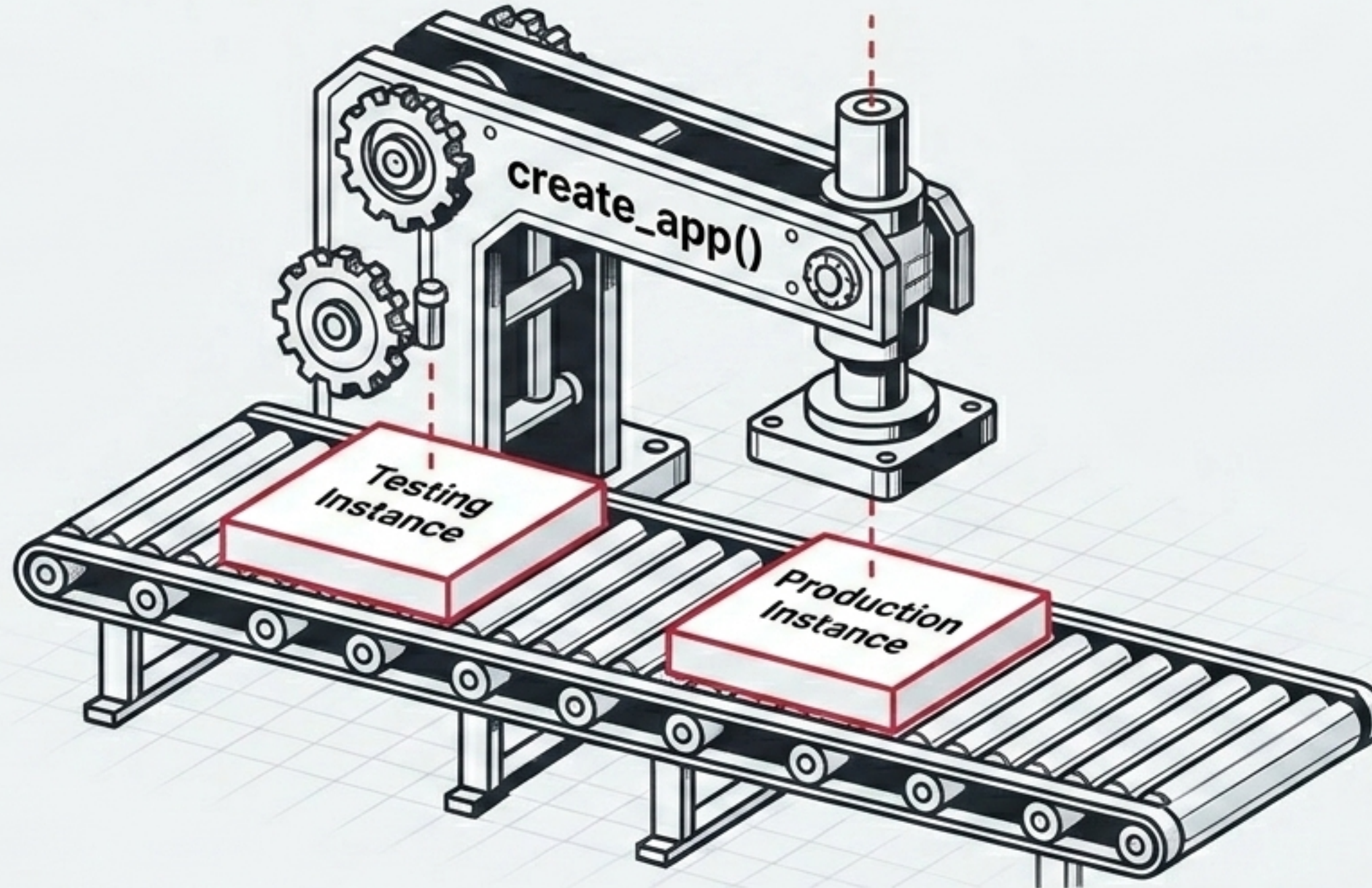
Keeping the stack Python all the way through.



Zero build steps. No frontend package managers.
Fully traceable from browser request to Gemini response.

The App Factory Pattern

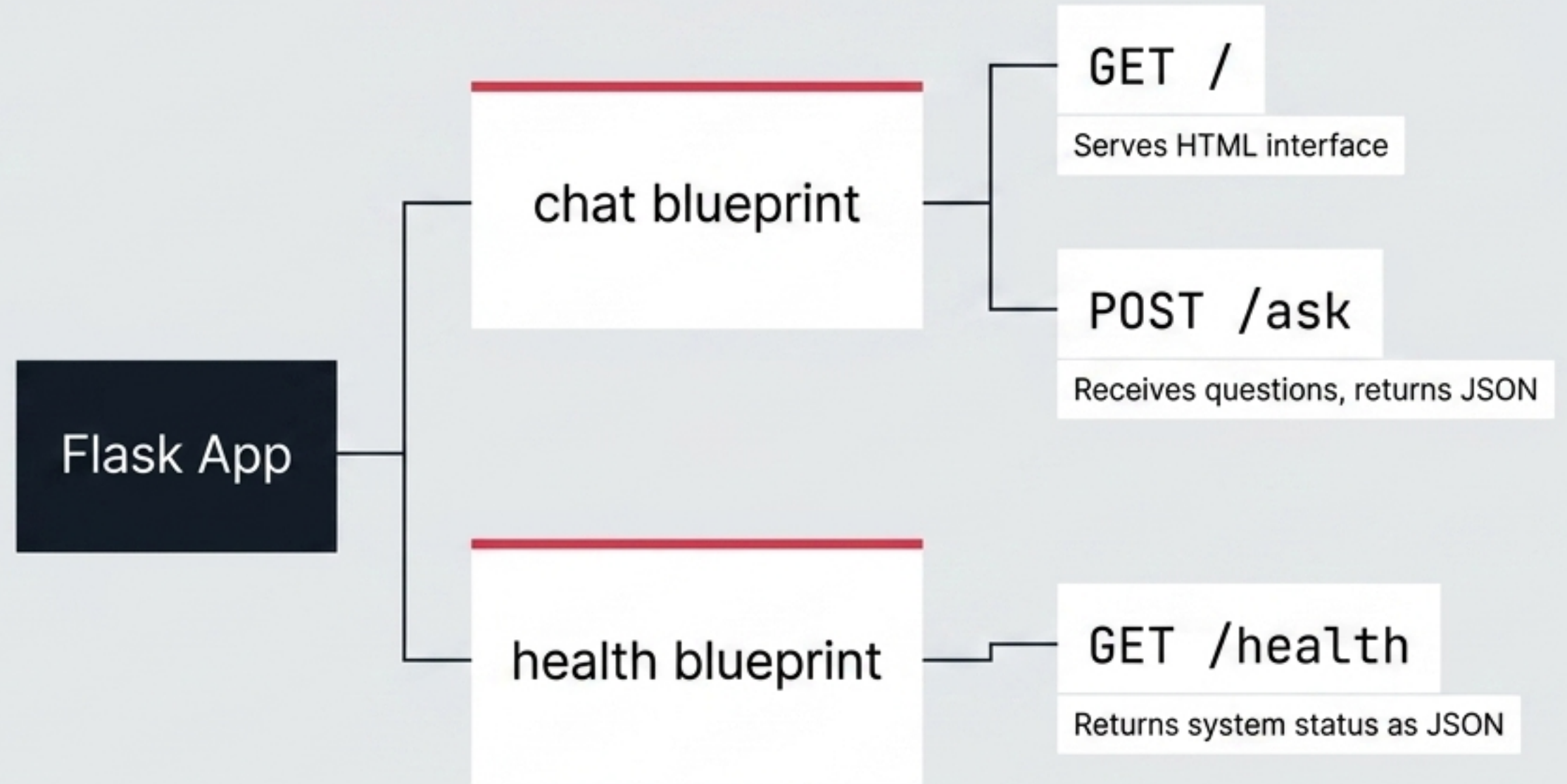
Standardizing instantiation to prevent circular errors.



`create_app()` guarantees a fresh app instance for isolated testing and clean blueprint registration.

Modular Routing with Blueprints

Splitting the codebase to keep the main application clean.



Enables independent development, testing, and future expansion (e.g., adding an admin panel).

Operations & Monitoring

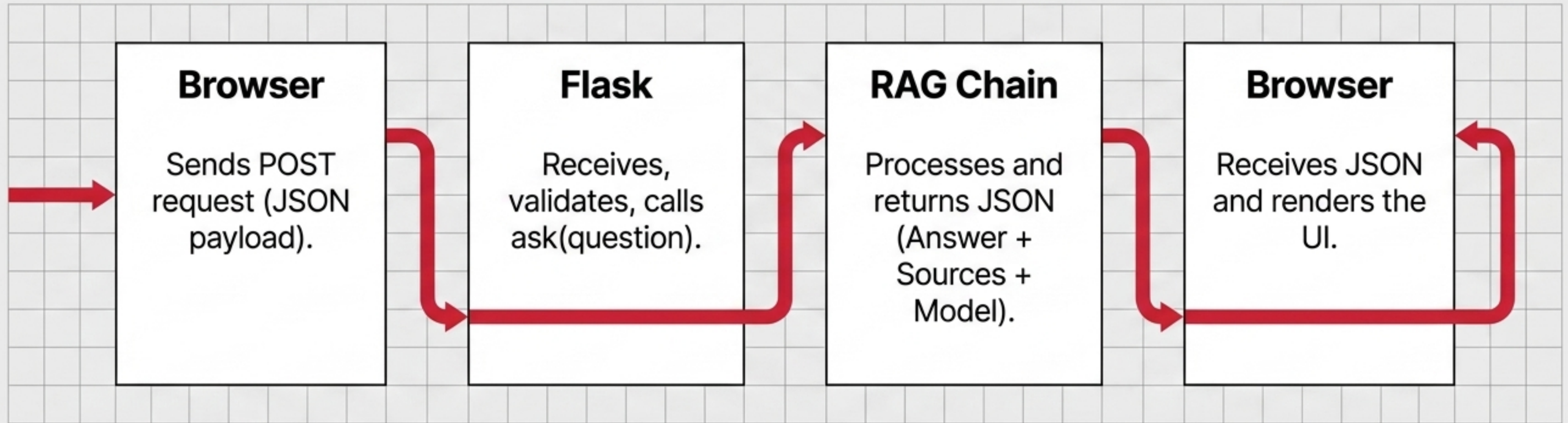
Engineers build systems for operability.

```
GET /health
{
  status: UP,
  faiss_index: Loaded,
  vectors_count: 1,420,
  models: Configured
}
```

Costs nothing to build; saves significant debugging time during operation.

The Core API Endpoint

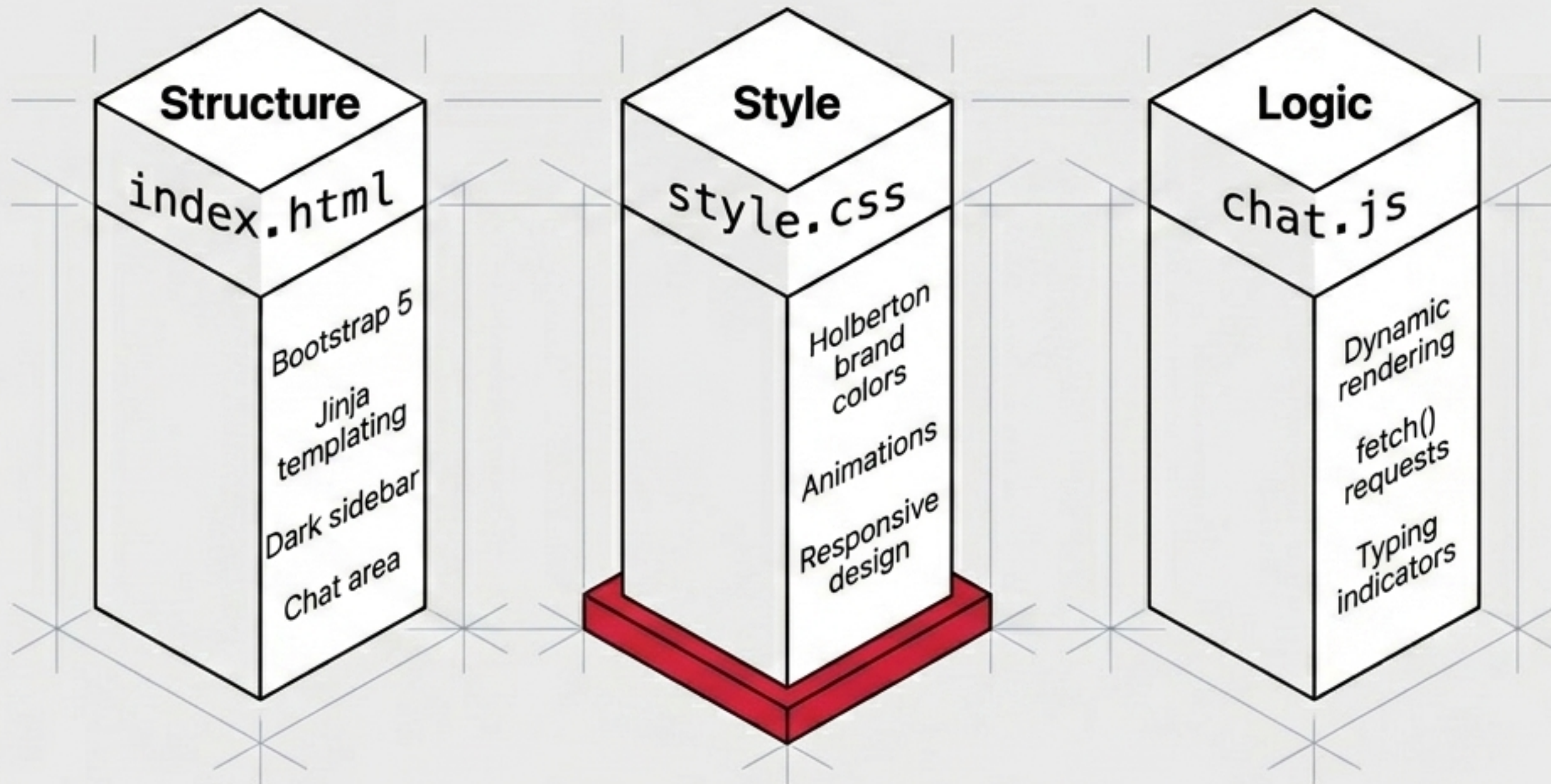
/ask returns raw data, never HTML.



This data-only separation is the foundational architecture of modern web applications.

The Frontend Triad

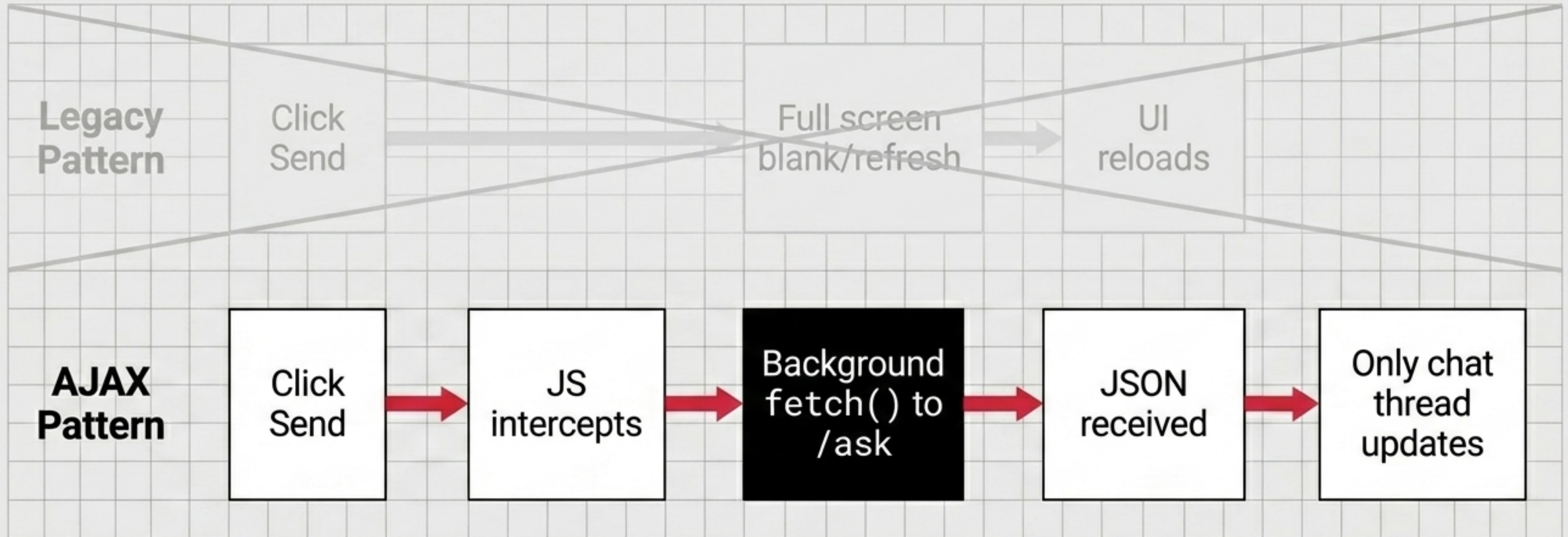
Three files, one seamless interface.



Clean, responsive layout separating structure, visual identity, and browser logic.

Seamless Interaction

The URL never changes; the page never flickers.



Utilizing modern `fetch()` patterns for a native-app feel, standard in tools like Slack and Gmail.

Styling with Purpose

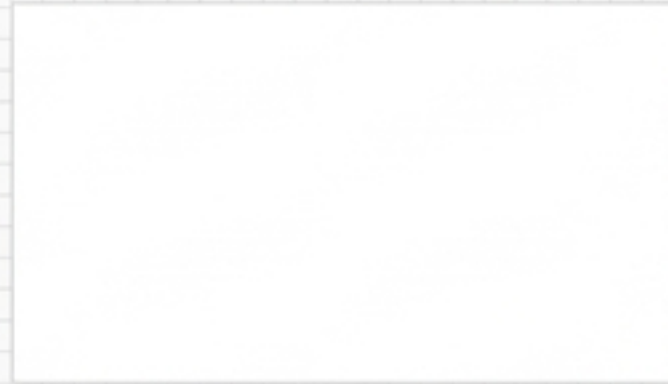
Extracted CSS is the professional standard.



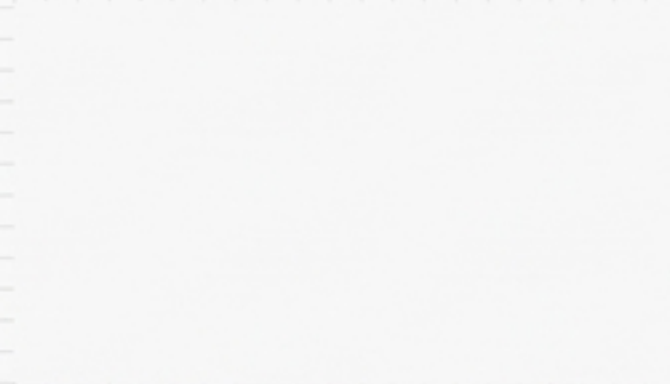
Cherry (#E31C3F)
Primary accent, buttons.



Midnight (#292728)
Sidebar, headings.



Ice White (#FFFFFF)
Content surfaces, bubbles.

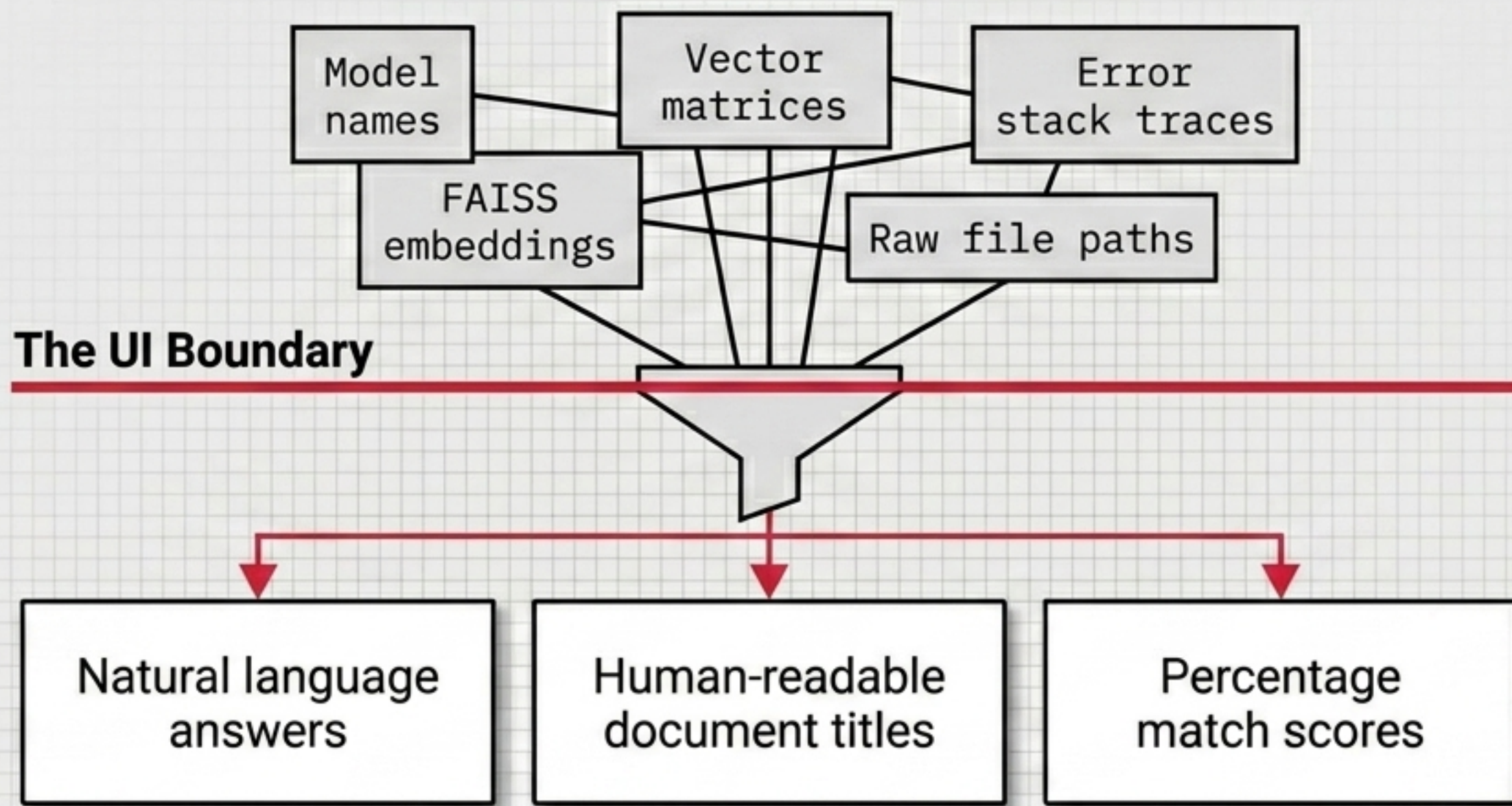


Light Gray (#F9F9F9)
Page background, inputs.

External stylesheets cache after the first load and enable seamless cross-page scaling. Avoid inline styles for production.

Filter Out the Developer

Design strictly for user clarity. Hide implementation complexity.



Every detail that leaks into the UI without helping the user is a failure of design.

Building Trust Through UI

Source citations separate trustworthy tools from black boxes.

Collapsed State

 Architecture_Guidelines.pdf 94% Match

Expanded State

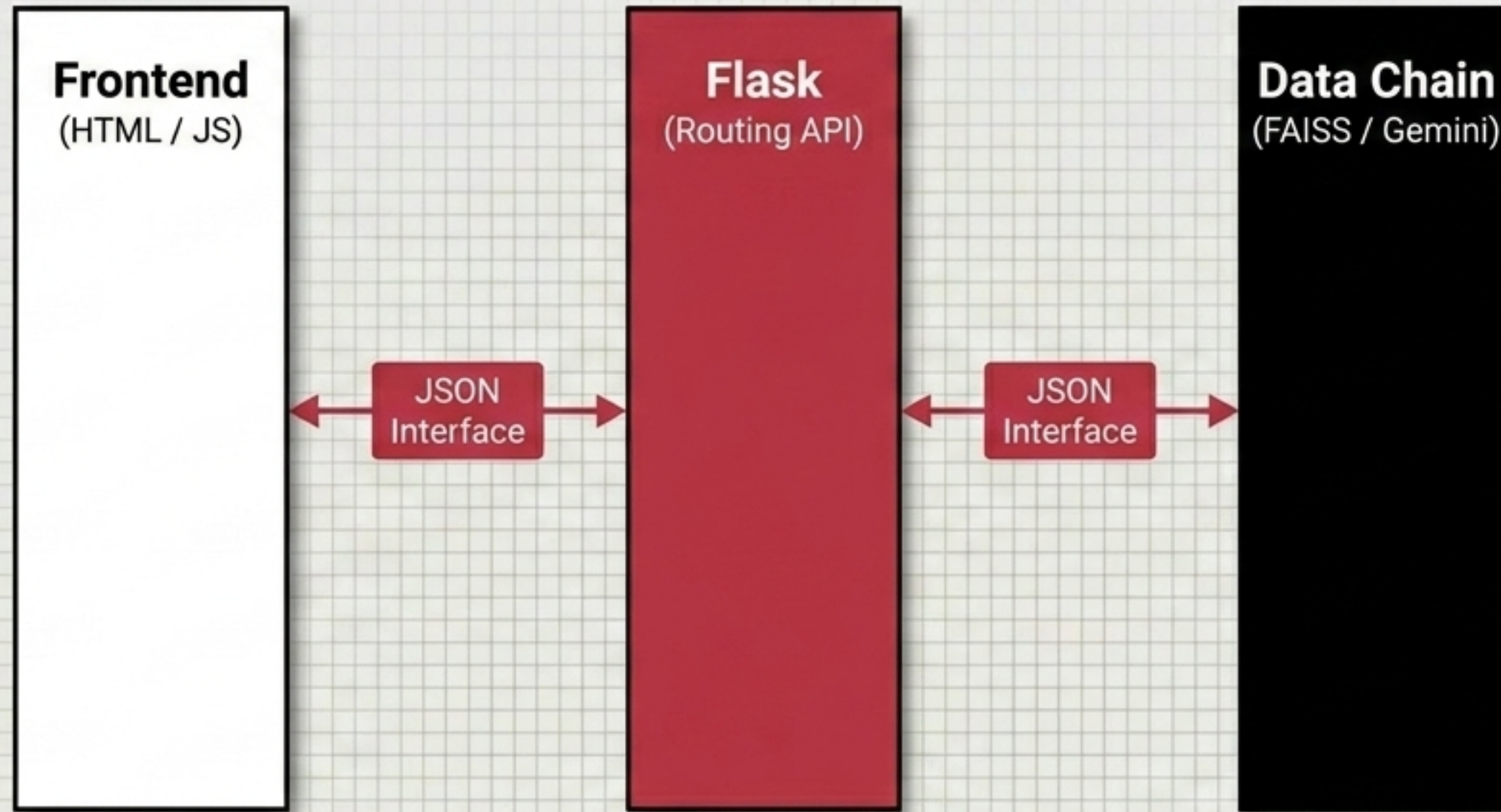
 Architecture_Guidelines.pdf Expand 

"The system architecture requires decoupling the frontend UI from the FAISS indexing service..."

Teaches information literacy by showing the system's work natively in the chat interface.

Separation of Concerns

Three blind layers, perfectly aligned.



The Frontend knows only HTML/JS. Flask knows only routing. FAISS/Gemini know only data. Easy to extend, easy to debug.

Three Lessons from Phase 3

Moving from building scripts to engineering products.



Architecture

Isolate responsibilities cleanly. Separation of Concerns ensures scalability.



Design

Hide backend implementation details. Prioritize user clarity and simplicity.



Product

The interface is what makes the project real for non-technical users.

The gap between a working terminal script and a useful product is almost entirely about the interface.